CS151 Data Structures

Quiz: 4 **Date: 10/8/26**
Name:

1.
```
void mystery(Queue Q) {
  Stack S = new Stack();

  while (!Q.isEmpty()) {
    S.push(Q.dequeue());
  }

  while (!S.isEmpty()) {
    Q.enqueue(S.pop());
  }
}
```
   What does `mystery` do?

2. Suppose a circular queue of capacity n–1 elements is implemented with an array of n elements in the following way. We maintain `FRONT` and `REAR` array index variables, where `FRONT` stores the front element index and `REAR` is 1 past the rear element of the queue (next insertion index). Initially, `REAR = FRONT = 0`. Note that this implemenation is slightly different from the array-based circular queue you learned in class, which tracks front and size instead.

   (a) Give the conditions to detect queue full and queue empty.

   (b) Why is the queue capacity $n-1$ and not n?

   (c) Write code for `enqueue` and `dequeue`.

3.
```
void mystery(int n) {
  IntQueue q = new IntQueue();
  q.enqueue(0);
  q.enqueue(1);
  for (int i = 0; i < n; i++) {
    int a = q.dequeue();
    int b = q.dequeue();
    q.enqueue(b);
    q.enqueue(a + b);
    System.out.println(a);
  }
}
```

Assume that `IntQueue` is a queue storing integers. What does `mystery` do?

4.
```
Function Insert(Q, x):
  S1.push(x)

Function Delete(Q):
  If stack S2 is empty:
    While stack S1 is not empty:
      S2.push(S1.pop())
  Return S2.pop()
}
```

An implemenation of a queue `Q` using two stacks `S1` and `S2` is given above in pseudo-code (without empty/full exception handling). You should be familiar with this because you were asked to implement a two-stack queue in your homework. If we perform n insertions and $m(<n)$ deletions in arbitrary order on an empty `Q` of this kind, what is the min and max number of push and pop operations performed respectively in the process, in terms of m and n?